CS 24302 : COMPILER DESIGN LABORATORY

Lab Session 8 : To be Decided

Scheduled : 3rd week of October 2026 (To be rescheduled in case of holiday)

Faculty Team : Bikas Kanti Sarkar, Indrajit Mukherjee, Prashant Pranav, Mohd. Shah Fahad, Supreeti Kamilya and Supratim Biswas (in absentia)

Research Scholar : Swarna Aishwarya Twinkle

Recap of Lab 7:
1. Grammar writing for declaration of scalar variables of C.
2. Creation of a simple symbol table structure to save input declarations.
3. Semantic analysis that detects errors in declarations using the symbol table of item 2 above.

Objective :

- Extend the assignment of Lab 6 to write a grammar for assignment statement of C with expressions involving a reasonable subset of operators.
- Perform semantic analysis and generation of Three Address Code (TAC) for assignment statement(s) assuming all identifiers are of type “int” so that type checking issues are ignored initially.
- Add “float” data type with “int”, handle type check and type conversion issues as performed by gcc before generating TAC.
- In case of semantic error in expression, discontinue generation of TAC but continue parsing till end of input.

Handout : Read the "Handout on Semantic Analysis and Intermediate Code Generation for Assignment Statement " for the structure and semantics of intermediate code (IC) statements to be used for this session.

P1. The scripts, "asgn1.l" and "asgn1.y" use an ambiguous grammar with disambiguating rule of yacc to generate one or more assignment statements in C, using ID as operand and operators {+ =} and delimiter ‘;’. The semantic actions, as discussed in theory, are outlined below.

1,2) S → S A | A
3) A → ID = E ; {gencode(ID.name ':=' E.place}
4, 5) E → E_1 + E_2 {E.place = newtemp(); gencode (E.place ':=' E_1.place '+' E_2.place);}
| ID {E.place = newtemp(); gencode (E.place ':=' ID.place);}

A possible implementation of the above SDTS is given in the scripts. The passing of the lexeme for an ID recognized by the scanner is achieved by i) defining union{} in the yacc scripts and ii) by assigning to yylval the lexeme saved in yytext. In the yacc script, we need to write functions in C for gencode(), newtemp() and genlabel(); these functions have been written in a separate file, "tac-functions.c". You may use this file or choose to replace by your own implementation. Note that file i/o operations are used to append generated TAC to a file named as, "codefile". The semantic actions use the $-variables to hold and propagate semantic attributes.

(a) Generate the TAC generator from the given scripts and test its performance for the input, "asgninp1.txt". Analyse all the outputs produced and write a report justifying each line of the outputs.

(b) Construct another valid input for the given SDTS and test and report your finding with reasons on the working of the TAC generator.

(c) Conduct 2 more experiments, one by removing the line, "%left '+' ", from the yacc script; and the other by replacing "%left '+' " by "%right '+' ". For each of these two changes, reconstruct the generators and run them over the same input file. Summarize your findings in the lab report.

P2. Extend the set of operators given in P1 with the inclusion of binary ops, {–, *, /, %, &&, | |} and operands to include {NUM, true, false}beside ID. Design a parser that is consistent with C semantics(see Appendix for operator properties). Two sample inputs and their desirable outputs are given as illustration. You may have to augment both the scripts to solve this problem.

Input 1 ("asgn2inp1.txt")	Input 2 ("asgn2inp2.txt")

100
x1 = 10;
x2 = x1 - 25;
x3 = x1 * x2 && 15 / x2 || false;
	100
	x1 = 10;
	x2 = x1 - 25;
	x3 = x1 * x2 || 15 / x2 && false;

Output1	Output2
100: t1 := 10 101: x1 := t1 102: t2 := x1 103: t3 := 25 104: t4 := t2 - t3 105: x2 := t4 106: t5 := x1 107: t6 := x2 108: t7 := t5 * t6 109: t8 := 15 110: t9 := x2 111: t10 := t8 / t9 112: t11 := t7 && t10 113: t12 := false 114: t13 := t11 || t12 115: x3 := t13	100: t1 := 10 101: x1 := t1 102: t2 := x1 103: t3 := 25 104: t4 := t2 - t3 105: x2 := t4 106: t5 := x1 107: t6 := x2 108: t7 := t5 * t6 109: t8 := 15 110: t9 := x2 111: t10 := t8 / t9 112: t11 := false 113: t12 := t10 && t11 114: t13 := t7 || t12 115: x3 := t13

(a) Construct a grammar for P2 and add actions in the form of SDTS, using the format of theory, and write it in your lab file, before proceeding for implementation.

(b) Write the modified lex and yacc scripts to write SDTS for a grammar equivalent to the following.

$E \rightarrow E_1 + E_2 \mid E_1 * E_2 \mid E1 - E2 \mid E1 / E2 \mid E1 \% E2 \mid E1 \&\& E2 \mid E1 \mid\mid E2$
$\mid ID \mid NUM \mid TRUE \mid FALSE$

(c) Consider the file, "asgn2inp1.c" which is a C program with semantics equivalent to "asgn2inp1.txt". Compile and run the C program and save its result. Create a C program from the output output generated by your parser, compile and run this program and compare with the output of gcc.

P3 [Bonus Problem] Floating point data type is added to grammar now with operators {+, –, *, /, %} and operands NUM and ID. Since both integer and floating point numbers coexist in the grammar, issues of type checking and type conversion have to be dealt and handled. Declarations are now part of the grammar and a symbol table is required to save the attributes of identifiers (arrays and other user defined types are outside the scope). A comprehensive sample grammar for P3 follows.

Prog → M dlist S { } M → ε {initialize(symtab);}
dlist → $dlist_1$ decl ; { } | decl ; { }
decl → type ID { if ID exists in symtab then issue error else insert ID with type attributes in symtab;
propagate type attributes to decl; }
| $decl_1$, ID {if ID exists in symtab then issue error else insert ID with $decl_1$ attributes in symtab;
propagate $decl_1$ attributes to decl;}
type → INT {propagate attributes of INT to type} | FLOAT {propagate attributes of FLOAT to type}
S → S_1 A { } | A { }
A → ID = E ; {if ID is valid then
if ID.type is same as E.type then create a temporary and generate TAC for assignment
else { create a temporary to convert “int” operand to “float” type; generate TAC for assignment}
else issue semantic error }

E $\rightarrow E_1$ % E_2 {if (E_1.type == “float” OR E_2. Type == “float”) issue error message;
else create a temporary to generate TAC; assign attributes of E}
E $\rightarrow E_1$ OP E_2 { if (both types are “int”)
then {t := E1.place INT-OP E2.place; assign attributes of E;}
else if (E1 is int and E2 is “float”)
then {t1 := i2f (E1.place); t2 := t1 FLOAT-OP E2.place;}
…. similar code for other case }
E → NUM { } | REAL { } | ID { }
Two built-in type conversion functions, i2f() and f2i(), to convert an int operand to float and conversely are made available to you. TAC for sample inputs is provided to you for illustration. Two C files are given for symbol table creation and TAC generation.

Sample Input 1 (“mixed-mode-inp1.txt”)	Sample Input2 (“mixed-mode-inp2.txt”)

int a, b, c;
float x, y, z;
int res;
a = 10; b = 20; x = 2.5; y = 12.5; z = 15.5; c= 30;
res = a * x + b % 10 * y - b/c/x;
	int a, b, c;
	float x, y, z;
	int res;
	a = 10; b = 20; x = 2.5; y = 12.5; z = 15.5; c= 30;
	res = a * x + b % y * 10 - b/c/x;

Sample Output 1 (symbol table is omitted)	**Sample Output2 (symbol table is omitted)**
10: t1 := 10 11: a := t1 12: t2 := 20 13: b := t2 14: t3 := 2.5 15: x := t3 16: t4 := 12.5 17: y := t4 18: t5 := 15.5 19: z := t5 20: t6 := 30 21: c := t6 22: t7 := a 23: t8 := x 24: t9 := i2f t7 25: t10 := t9 float* t8 26: t11 := b 27: t12 := 10 28: t13 := t11 % t12 29: t14 := y 30: t15 := i2f t13 31: t16 := t15 float* t14 32: t17 := t10 float+ t16 33: t18 := b 34: t19 := c 35: t20 := t18 int/ t19 36: t21 := x 37: t22 := i2f t20 38: t23 := t22 float/ t21 39: t24 := t17 float- t23 40: t25 := f2i t24 41: res := t25	10: t1 := 10 11: a := t1 12: t2 := 20 13: b := t2 14: t3 := 2.5 15: x := t3 16: t4 := 12.5 17: y := t4 18: t5 := 15.5 19: z := t5 20: t6 := 30 21: c := t6 22: t7 := a 23: t8 := x 24: t9 := i2f t7 25: t10 := t9 float* t8 26: t11 := b 27: t12 := y Error: invalid operands to binary mod (have 'int’& 'float') Intermediate code generation is suspended but parsing continues.

(a) Write scripts for designing a TAC generator for this problem.
(b) Run the generator with both the input files and write the TAC generated for each input with reasons in your report.

End of the Experiment 8